

University of New Brunswick

Faculty of Engineering



ECE 2215

Final Project Report

December 3rd, 2024

Adam Bateman 3753751 (25%)

A handwritten signature in black ink that reads "Adam Bateman".

Ahmed Hessein Hassona 3772324 (25%)

A handwritten signature in black ink that reads "Ahmed Hessein Hassona".

Alexander Ross 3752371 (25%)

A handwritten signature in black ink that reads "Alexander Ross".

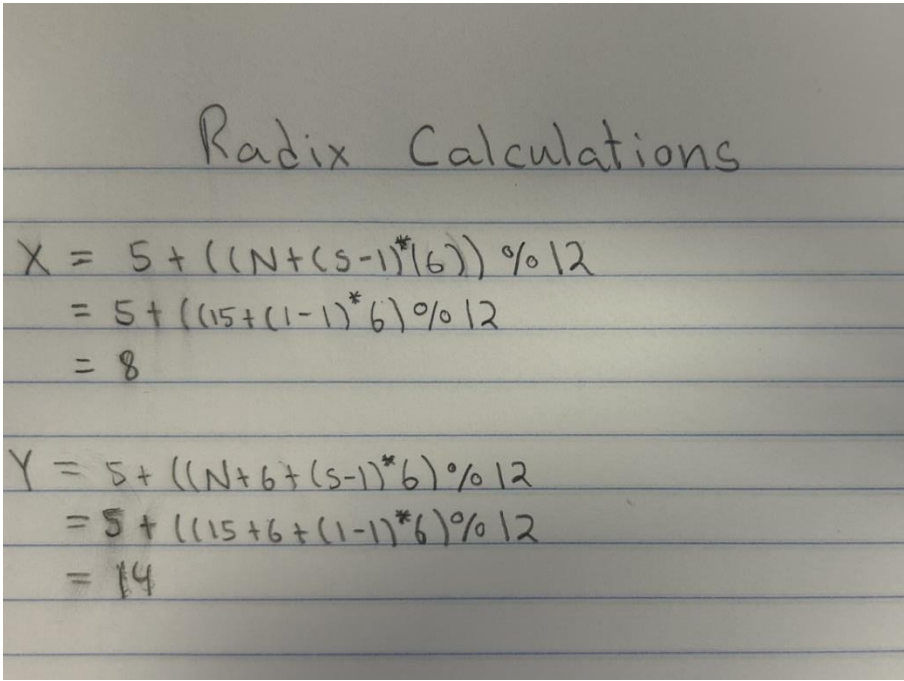
Corey White 3759566 (25%)

A handwritten signature in black ink that reads "Corey White".

Introduction

The project task is to create a modulo 2345 counter with respect to a radix Y. In our case, our radix Y is 14. This counter must have functionalities allowing it to swap between radix Y and X as an output, as well as being capable of counting both up and down. The counter must be able to accept a load value only if it is less than the max value allowed and have a switch to enable/disable the counter. The load value will be displayed to the red LEDs in binary, and the output is sent to the 7-segment displays in the radix selected. The green LED's will display the counting value of the lowest 8 bits.

Radix



Radix Calculations

$$\begin{aligned} X &= 5 + ((N + (S-1) * 6)) \% 12 \\ &= 5 + ((15 + (1-1) * 6)) \% 12 \\ &= 8 \end{aligned}$$
$$\begin{aligned} Y &= 5 + ((N + 6 + (S-1) * 6)) \% 12 \\ &= 5 + ((15 + 6 + (1-1) * 6)) \% 12 \\ &= 14 \end{aligned}$$

Design

Top Level Entity Code:

```
ownloads/final_project_restored/fullcounter - fullcounter
ssing Tools Window Help

toplevel.vhd* x clk_gen_1_output.vhd x fullcount.vhd x r8r14.vhd x radix14_to_7seg.vhd x octal_to_7seg.vhd x

1 --top level design file
2 --radix means what we want to output, 0 is 8, 1 is 14
3 library ieee;
4 use ieee.std_logic_1164.all;
5
6 entity topLevel is
7     port(
8         updown, enable, radix, load, clock: in std_logic;
9         parallel_inputs : in std_logic_vector(12 downto 0);
10        badload : out std_logic;
11        seg7_1, seg7_2, seg7_3, seg7_4, seg7_5: out std_logic_vector(6 downto 0);
12        loaddisplay : out std_logic_vector(12 downto 0);
13        lsbdisplay : out std_logic_vector(7 downto 0)
14    );
15 end entity;
16
17 architecture logic of topLevel is
18
19     signal c: std_logic;
20     signal q: std_logic_vector(12 downto 0);
21     signal qn: std_logic_vector(12 downto 0);
22     signal qnew: std_logic_vector(15 downto 0);
23     signal seg7_18, seg7_28, seg7_38, seg7_48, seg7_58: std_logic_vector(6 downto 0);
24     signal seg7_114, seg7_214, seg7_314, seg7_414, seg7_514: std_logic_vector(6 downto 0);
25
26     component fullcount is
27         port(clock, ena, updown, isload : in std_logic;
28             pininput : in std_logic_vector(12 downto 0);
29             inlim : out std_logic;
30             qpout, qnpout : out std_logic_vector(12 downto 0));
31     end component fullcount;
32
33     component r8r14 is
34         port(input : in std_logic_vector(12 downto 0);
35             output : out std_logic_vector(15 downto 0));
36     end component r8r14;
37
38     component octal_to_7seg is
39         port(C,B,A : in std_logic;
40             HL: out std_logic_vector(6 downto 0));
41     end component octal_to_7seg;
42
43     component radix14_to_7seg is
44         port(D,C,B,A : in std_logic;
45             HL: out std_logic_vector(6 downto 0));
46     end component radix14_to_7seg;
47
48     component clk_gen_1_output is
49         generic(n : integer := 25000;
50             nl : integer := 1000);
51         port(clock : in std_logic;
52             c_out : out std_logic);
53     end component clk_gen_1_output;
54
55 begin
56
57     stage0: clk_gen_1_output port map(clock, c);
58     stage1: fullcount port map(c, enable, updown, not(load), parallel_inputs, badload, q, qn); --change clock to c
59     stage2: r8r14 port map(q, qnew);
60
61     --green 1sb lights
62     lsbdisplay(7 downto 0) <= q(7 downto 0);
63
64     --display the loading value
65     loaddisplay <= parallel_inputs;
66
67     stage3: octal_to_7seg port map(q(2), q(1), q(0), seg7_18);
68     stage4: octal_to_7seg port map(q(5), q(4), q(3), seg7_28);
69     stage5: octal_to_7seg port map(q(8), q(7), q(6), seg7_38);
70     stage6: octal_to_7seg port map(q(11), q(10), q(9), seg7_48);
71     stage7: octal_to_7seg port map(q(0), q(12), seg7_58);
72     stage8: radix14_to_7seg port map(qnew(3), qnew(2), qnew(1), qnew(0), seg7_114);
73     stage9: radix14_to_7seg port map(qnew(7), qnew(6), qnew(5), qnew(4), seg7_214);
74     stage10: radix14_to_7seg port map(qnew(11), qnew(10), qnew(9), qnew(8), seg7_314);
75     stage11: radix14_to_7seg port map(qnew(15), qnew(14), qnew(13), qnew(12), seg7_414);
76     stage12: radix14_to_7seg port map('0', '0', '0', '0', seg7_514);
77
78     process(radix)
79     begin
80         if radix = '0' then
81             seg7_1 <= seg7_18;
82             seg7_2 <= seg7_28;
83             seg7_3 <= seg7_38;
84             seg7_4 <= seg7_48;
85             seg7_5 <= seg7_58;
86         else
87             seg7_1 <= seg7_114;
88             seg7_2 <= seg7_214;
89             seg7_3 <= seg7_314;
90             seg7_4 <= seg7_414;
91             seg7_5 <= seg7_514;
92         end if;
93     end process;
94
95 end architecture;
```

The top level was used to tie all files together as a whole. The top level has a full counter, a radix 8 to radix 14 converter, 2 seven segment converters, one for each radix, and a clock controller as well.

Instantiated Code:

Fullcount.vhd :

```
Downloads/final_project_restored/fullcounter - fullcounter
ssing Tools Window Help

toplevel.vhd x fullcount.vhd x clk_gen_1_output.vhd x r8r14.vhd x radix14_to_7seg.vhd x octal_to_7seg.vhd x Compilation Report - fullcounter x

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity fullcount is
5 port(clock, ena, updown, isload : in std_logic;
6 pininput : in std_logic_vector(12 downto 0);
7 inlim : out std_logic;
8 qpout, qnpout : out std_logic_vector(12 downto 0)); -- do we need q1 in octalcount?
9 end entity;
10
11 architecture brains of fullcount is
12
13 signal qt : std_logic_vector(12 downto 0);
14 signal qcarryint : std_logic_vector(3 downto 0);
15 signal clear : std_logic_vector(12 downto 0);
16 signal chold : std_logic;
17 signal preset : std_logic_vector(12 downto 0);
18 signal phold : std_logic;
19 signal l : std_logic;
20
21 component octalcount is
22 port(clk, en, ud, load : in std_logic;
23 CR : in std_logic_vector(2 downto 0);
24 PR : in std_logic_vector(2 downto 0);
25 data : in std_logic_vector(2 downto 0);
26 qo : out std_logic_vector(2 downto 0);
27 qcarry : out std_logic);
28 end component octalcount;
29
30 component onecount is
31 port(clk, en, load, CR, PR : in std_logic;
32 data : in std_logic;
33 qo : out std_logic);
34 end component onecount;
35
36 begin
37
38 process (pininput)
39 begin
40 if pininput > "1011111111000" then
41 inlim <= '1';
42 l <= '0';
43 else
44 inlim <= '0';
45 l <= isload;
46 end if;
47 end process;
48
49 phold <= (qt(0) or qt(1) or qt(2) or qt(3) or qt(4) or qt(5) or qt(6) or qt(7) or qt(8) or qt(9) or qt(10) or qt(11) or qt(12));
50 chold <= not(qt(3) and qt(4) and qt(5) and qt(6) and qt(7) and qt(8) and qt(9) and qt(10) and qt(11) and qt(12));
51
52 process (updown)
53 begin
54 if updown = '0' then
55 clear(0) <= chold; clear(1) <= chold; clear(2) <= chold; clear(3) <= chold; clear(4) <= chold; clear(5) <= chold; clear(6) <= chold;
56 clear(7) <= chold; clear(8) <= chold; clear(9) <= chold; clear(10) <= chold; clear(11) <= chold; clear(12) <= chold;
57 preset <= "111111111111";
58 else
59 clear(10 downto 3) <= "11111111"; clear(12) <= '1';
60 clear(0) <= phold; clear(1) <= phold; clear(2) <= phold; clear(11) <= phold;
61 preset(2 downto 0) <= "111"; preset(11) <= '1';
62 preset(3) <= phold; preset(4) <= phold; preset(5) <= phold; preset(6) <= phold; preset(7) <= phold;
63 preset(8) <= phold; preset(9) <= phold; preset(10) <= phold; preset(12) <= phold;
64 end if;
65 end process;
66
67 stage0: octalcount port map(clock, ena, updown, l, clear(2 downto 0), preset(2 downto 0), pininput(2 downto 0), qt(2 downto 0), qcarryint(0));
68 stage1: octalcount port map(clock, qcarryint(0), updown, l, clear(5 downto 3), preset(5 downto 3), pininput(5 downto 3), qt(5 downto 3), qcarryint(1));
69 stage2: octalcount port map(clock, qcarryint(1), updown, l, clear(8 downto 6), preset(8 downto 6), pininput(8 downto 6), qt(8 downto 6), qcarryint(2));
70 stage3: octalcount port map(clock, qcarryint(2), updown, l, clear(11 downto 9), preset(11 downto 9), pininput(11 downto 9), qt(11 downto 9), qcarryint(3));
71 stage4: onecount port map(clock, qcarryint(3), l, clear(12), preset(12), pininput(12), qt(12));
72
73 qpout <= qt;
74 qnpout <= not(qt);
75
76 end architecture brains;
```

This is the full counter, which takes the octal counter and one bit counter as components. This allows for counting in radix 8, with the 4 octal counters counting the first 12 bits, and the one counter counts the 13th bit. This file also contains the input for the load value as well.

R8r14.vhd:

```
Downloads/final_project_restored/fullcounter - fullcounter
essing Tools Window Help

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4
5  entity r8r14 is
6  port(input : in std_logic_vector(12 downto 0);
7        output : out std_logic_vector(15 downto 0));
8  end entity;
9
10 architecture logic of r8r14 is
11
12     signal carry : std_logic_vector(12 downto 0);
13     signal carry1 : std_logic_vector(12 downto 0);
14     signal carry2 : std_logic_vector(12 downto 0);
15     signal carry3 : std_logic_vector(12 downto 0);
16
17     component divider is
18     port(denom      : IN STD_LOGIC_VECTOR (3 DOWNTO 0);
19          numer      : IN STD_LOGIC_VECTOR (12 DOWNTO 0);
20          quotient   : OUT STD_LOGIC_VECTOR (12 DOWNTO 0);
21          remain     : OUT STD_LOGIC_VECTOR (3 DOWNTO 0));
22     end component;
23
24     begin
25
26
27     stage0: divider port map("1110", input(12 downto 0), carry(12 downto 0), output(3 downto 0));
28     stage1: divider port map ("1110", carry(12 downto 0), carry1(12 downto 0), output(7 downto 4));
29     stage2: divider port map ("1110", carry1(12 downto 0), carry2, output(11 downto 8));
30     stage3: divider port map ("1110", carry2, carry3, output(15 downto 12));
31
32
33
34     end architecture;
35
```

This file converts from radix 8, which is the counting radix, to radix 14, which is the alternate radix to output as. It uses a divider as a component, which is a library used to facilitate the conversion.

Octal_to_7seg.vhd:

```
Downloads/final_project_restored/fullcounter - fullcounter
essing Tools Window Help

toplevel.vhd fullcount.vhd r8r14.vhd octal_to_7seg.vhd clk_gen_1_output.vhd radix14_to_7seg.vhd Compilation Report - fullcounter

1 library ieee;
2 use ieee.std_logic_1164.all;
3 entity octal_to_7seg is
4 port(
5     C,B,A : IN std_logic;
6     HL : OUT std_logic_vector(6 downto 0)
7 );
8 end octal_to_7seg;
9 architecture decoder of octal_to_7seg is
10 signal m0,m1,m2,m3,m4,m5,m6,m7 : std_logic;
11
12 begin
13
14     m0 <= (not(C) and not(B) and not(A));
15     m1 <= (not(C) and not(B) and A);
16     m2 <= (not(C) and B and not(A));
17     m3 <= (not(C) and B and A);
18     m4 <= (C and not(B) and not(A));
19     m5 <= (C and not(B) and A);
20     m6 <= (C and B and not(A));
21     m7 <= (C and B and A);
22
23
24     HL(0) <= m1 or m4;
25     HL(1) <= m5 or m6;
26     HL(2) <= m2;
27     HL(3) <= m1 or m4 or m7;
28     HL(4) <= m1 or m3 or m4 or m5 or m7;
29     HL(5) <= m1 or m2 or m3 or m7;
30     HL(6) <= m0 or m1 or m7;
31 end decoder;
```

This is the code that converts each octal value into an output suitable to provide to a seven-segment display, allowing us to display the value that we are currently counting at.

Radix14_to_7seg.vhd:

```
Downloads/final_project_restored/fullcounter - fullcounter
essing Tools Window Help

toplevel.vhd fullcount.vhd r8r14.vhd octal_to_7seg.vhd radix14_to_7seg.vhd clk_gen_1_output.vhd Compilation Report - fullcounter

1 library ieee;
2 use ieee.std_logic_1164.all;
3 entity radix14_to_7seg is
4 port(
5     D,C,B,A : IN std_logic;
6     HL : OUT std_logic_vector(6 downto 0)
7 );
8 end radix14_to_7seg;
9 architecture decoder of radix14_to_7seg is
10 signal m0,m1,m2,m3,m4,m5,m6,m7 : std_logic;
11 signal m8,m9,m10,m11,m12,m13 : std_logic;
12
13 begin
14
15     m0 <= (not(D) and not(C) and not(B) and not(A));
16     m1 <= (not(D) and not(C) and not(B) and A);
17     m2 <= (not(D) and not(C) and B and not(A));
18     m3 <= (not(D) and not(C) and B and A);
19     m4 <= (not(D) and C and not(B) and not(A));
20     m5 <= (not(D) and C and not(B) and A);
21     m6 <= (not(D) and C and B and not(A));
22     m7 <= (not(D) and C and B and A);
23     m8 <= (D and not(C) and not(B) and not(A));
24     m9 <= (D and not(C) and not(B) and A);
25     m10 <= (D and not(C) and B and not(A));
26     m11 <= (D and not(C) and B and A);
27     m12 <= (D and C and not(B) and not(A));
28     m13 <= (D and C and not(B) and A);
29
30
31
32     HL(0) <= m1 or m4 or m11 or m12 or m13;
33     HL(1) <= m5 or m6 or m11 or m12;
34     HL(2) <= m2 or m12;
35     HL(3) <= m1 or m4 or m7 or m9 or m10;
36     HL(4) <= m1 or m3 or m4 or m5 or m7 or m9;
37     HL(5) <= m1 or m2 or m3 or m7 or m12 or m13;
38     HL(6) <= m0 or m1 or m7;
39 end decoder;
```

This is the code that converts each Radix-14 value into an output suitable to provide to a seven-segment display, allowing us to display the value that we are currently counting at.

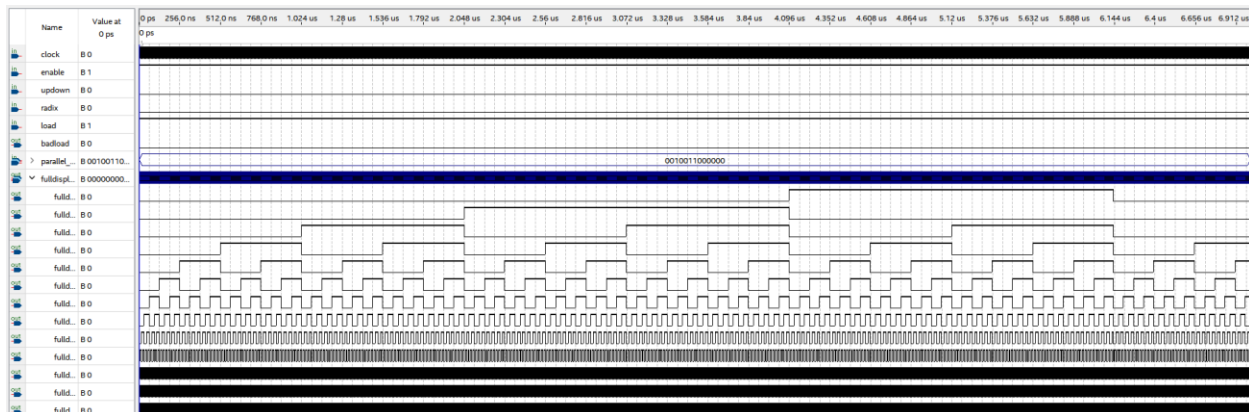
Clk_gen_1_output.vhd:

```
1  -- A down counter to generate a low frequency
2  -- clock for the ee2213 labs. The generic
3  -- variable n controls the output frequency.
4  -- The clock frequency is 5000000/((n-1)*(n1-1)) Hz.
5  -- The default frequency is 0.5 Hz.
6  -- 01/11/2017 - Upgraded to work with QUARTUS II - v17.0
7  -- E.C.G.
8
9  library ieee;
10 use ieee.std_logic_1164.all;
11 use ieee.std_logic_unsigned.all;
12
13 entity clk_gen_1_output is
14 generic( n : integer := 25000;
15          n1 : integer := 1000);
16 port( clock : in std_logic;
17       c_out : out std_logic);
18 end clk_gen_1_output;
19
20 architecture behavior of clk_gen_1_output is
21 signal count: integer := n;
22 signal scale: integer := n1;
23 signal c_val: std_logic := '0';
24
25 begin
26 process begin
27 wait until(rising_edge(clock));
28 if scale = 0 then
29 if count = 0 then
30 c_val <= not c_val;
31 count <= n - 1;
32 else
33 count <= count-1;
34 end if;
35 scale <= n1-1;
36 else
37 scale <= scale - 1;
38 end if;
39 c_out <= c_val;
40 end process;
41 end behavior;
```

This is the clock controller; it scales the timing on the clock so that it ticks at a slower rate making it much easier to observe the function of the counter.

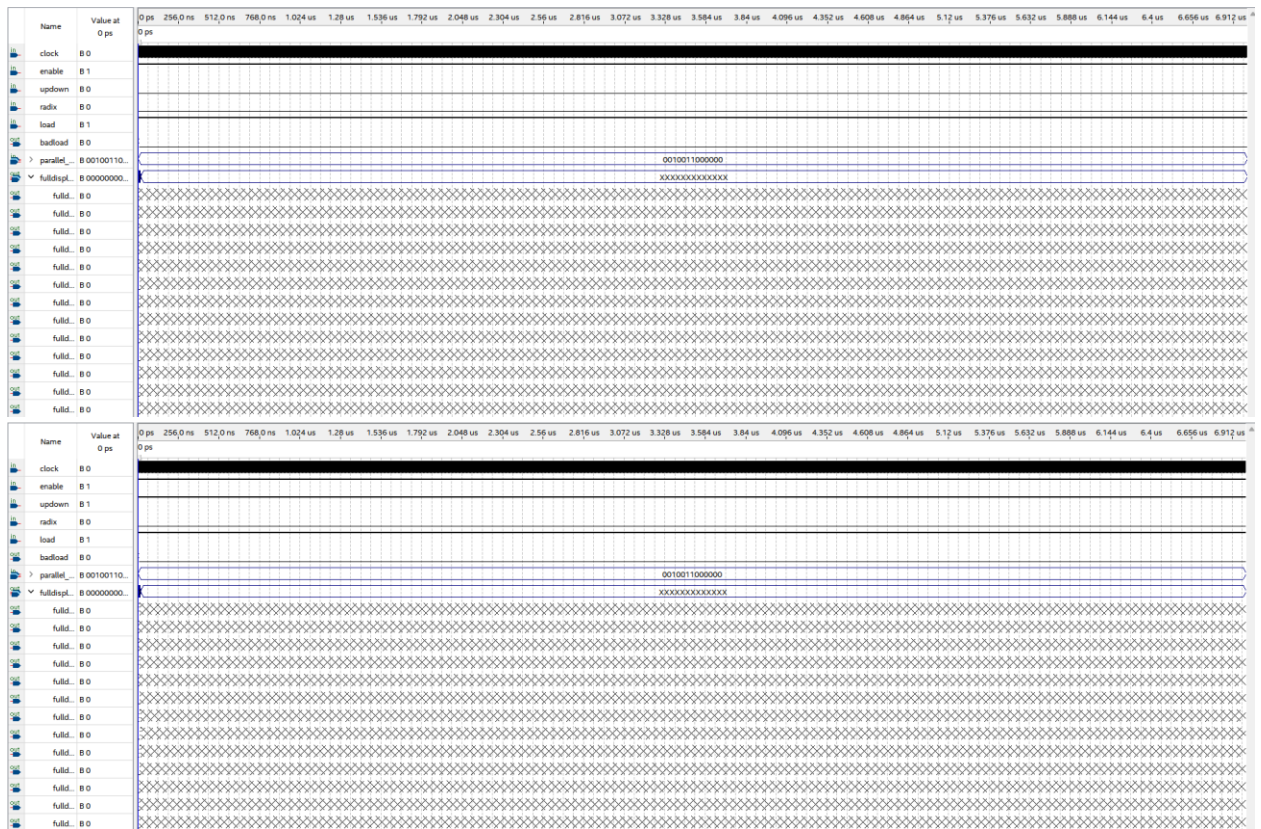
Results

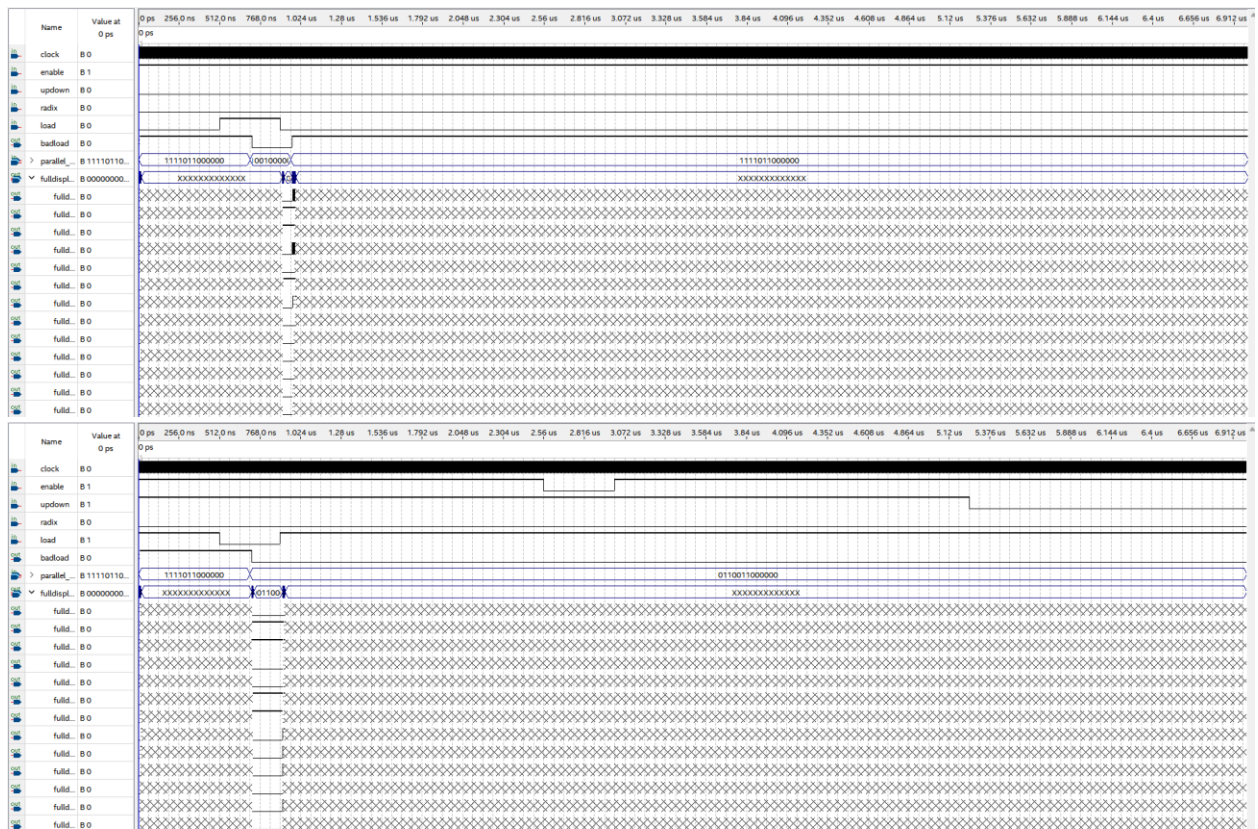
In these simulations a variable full count was added which shows all bits and the output from clk_gen_output was ignored.



This simulation shows the circuit counting up to its highest value resetting to 0.

Timing simulations for above cases





Everything in all simulations worked as expected.

Conclusion

This project required many VHDL files to be instantiated throughout and proved to be good practice using Quartus prime and instantiating files within files, creating a complex structure to perform a task more difficult than in a normal lab environment. This project also challenged us to work in a team larger than normal, which proved to be a great experience as well, since we were able to work as a team to tackle this larger problem.

Test Plan

- Display start/stop functionality of counter
 - Show that the counter will activate when the activation switch is toggled high
- Display the ability to swap between up and down counting
 - Show that the output can be swapped between an up counter and a down counter, with low up and high down
- Display the functionality to swap between radix 8 and radix 14
 - Show that the output radix can be toggled with radix 8 as low, and radix 14 as high
- Display functionality of 8 green LED's which displays the 8 lsb's
 - Show that the 8 green LED's will show the 8 lsb's of the value being counted
- Display functionality of 13 red LED's which display the load value, which is loaded via 13 load switches and a load push button
 - Show that the red LED's will display the load value that is represented by the 13 load input switches, which are all switches on the board
- Display functionality red LED which lights up if the load value on switches is too high
 - Enter a value that is too high and try to load it
- Display functionality of 7 segment display, which will display in the selected radix
 - Show that the current count value will be displayed on the 7-segment display